# The Vector–Graph–PRE–CAS Cognitive Architecture Family

Joachim De Witte

October 2026

©2026 Joachim De Witte. All rights reserved.
Published under the Creative Commons BY–NC–ND 4.0 license.

**Abstract**

Contemporary artificial intelligence systems are increasingly dominated by large-scale neural architectures that provide powerful emergent capabilities, but often at the cost of interpretability, controllability, traceability, and explicit state management. This paper introduces the *Vector–Graph–PRE–CAS (VGPC) Cognitive Architecture Family*, a modular architectural framework that combines neural inference with structured state spaces, deterministic transition mechanisms, reflective evaluation loops, and learnable behavioral control vectors.

In the VGPC family, cognitive states are represented as vectors, while a deterministic graph defines the admissible transitions between states. Neural components are used to infer target state directions within the vector space, after which optional modulation, rule-based constraints, and graph-guided decoding mechanisms determine concrete state transitions. The architecture is further extended with PRE (Propose–Retry–Evaluate) loops that enable recursive evaluation and refinement at multiple levels of abstraction, and with CAS (Content–Accuracy–Safety–Personality) vectors that provide learned, prompt-independent behavioral steering.

Rather than proposing a replacement for contemporary foundation models, VGPC is presented as a complementary cognitive architecture intended for hybrid intelligent systems. The framework supports the integration of neural, symbolic, and evaluative mechanisms within a unified computational model, allowing the construction of controllable and inspectable world engines, reasoning systems, memory systems, and domain-specific cognitive agents.

The paper defines the core concepts, design principles, and architectural variants of the VGPC family and discusses its role as a building block for future hybrid and agentic AI systems. Rather than introducing an isolated architecture, the VGPC framework generalizes principles already embodied by the GLP–Graph cognitive system and establishes a broader family of architectures for controllable, inspectable, and modular intelligence.

## 1 Introduction

Artificial intelligence has experienced unprecedented progress through the development of large-scale neural architectures, particularly foundation models and large language models. These systems demonstrate remarkable capabilities in language understanding, reasoning, planning, and knowledge retrieval. Nevertheless, their internal representations remain largely opaque, while properties such as long-term state consistency, explicit controllability, traceability, and structured behavioral steering remain challenging.

Historically, artificial intelligence has oscillated between neural and symbolic paradigms. Purely symbolic systems offer transparency, deterministic reasoning, and explicit state representations, but often struggle with adaptability and learning. Purely neural systems excel at learning from data and producing emergent behavior, but may sacrifice interpretability, reproducibility, and explicit control. As intelligent systems become increasingly autonomous and agentic, there is growing interest in hybrid approaches that combine the strengths of both perspectives.

Within the broader Hybrid Intelligence methodology developed in the Triadic Cosmos ecosystem, intelligence is viewed not as the consequence of a single algorithmic mechanism, but as the result of interacting cognitive subsystems that may employ different computational principles. In such a framework, neural, symbolic, evaluative, and memory-oriented components can coexist and cooperate as part of a larger intelligent architecture.

This paper introduces the *Vector–Graph–PRE–CAS (VGPC) Cognitive Architecture Family*, a general architectural framework designed around this hybrid philosophy. The framework combines vector-based state representations, graph-constrained state transitions, neural state-vector inference, recursive evaluation loops, and learnable behavioral control mechanisms. The resulting architecture aims to provide a balance between adaptability and control, allowing systems to benefit from learned representations while maintaining explicit and inspectable state spaces.

The VGPC architecture family did not emerge as a purely theoretical proposal, but as the abstraction of principles that became apparent during the development of concrete systems. In particular, the previously introduced GLP–Graph architecture can be viewed as a specific realization of the broader VGPC framework. The present work generalizes these underlying concepts into a unified architectural family capable of supporting a wider range of cognitive systems, world engines, reasoning engines, memory systems, and hybrid agentic architectures.

Importantly, VGPC is not proposed as a replacement for contemporary foundation models. Rather, it is intended as a complementary architectural component that can operate either as a standalone cognitive system within bounded domains or as part of larger hybrid intelligent systems. In such environments, VGPC-based subsystems may provide structured world modeling, state management, traceability, and behavioral control, while other components, including large language models, contribute broad generalization capabilities and natural language competence.

The remainder of this paper formalizes the VGPC architecture family, discusses its relation to existing work in cognitive architectures and neuro-symbolic AI, and outlines its role as a potential building block for future modular and hybrid intelligent systems.

## 2 Related Work

The Vector–Graph–PRE–CAS (VGPC) architecture family builds upon ideas that have appeared across multiple traditions within artificial intelligence, cognitive architectures, neural learning systems, graph-based models, and hybrid symbolic reasoning. Rather than belonging exclusively to any single tradition, VGPC combines concepts from several research directions into a unified architectural framework.

### 2.1 Cognitive Architectures

Classical cognitive architectures such as SOAR and ACT–R were designed to model cognitive processes through explicit state representations, symbolic reasoning mechanisms, procedural knowledge, and memory systems. These architectures emphasize interpretability, modularity, and the decomposition of intelligence into interacting subsystems.

VGPC shares the objective of constructing modular cognitive systems rather than end-to-end predictive models. However, unlike traditional symbolic cognitive architectures, VGPC employs vector-based state representations and neural inference mechanisms while retaining explicit state transitions and evaluation processes. The architecture therefore occupies an intermediate position between classical cognitive architectures and modern neural systems.

### 2.2 Large Language Models and Transformer Architectures

Transformer-based architectures and large language models have demonstrated remarkable capabilities through large-scale self-supervised learning. Knowledge, reasoning patterns, and behavioral tendencies are represented implicitly within distributed parameter spaces, enabling highly flexible and general-purpose behavior.

VGPC differs fundamentally from this paradigm. Instead of directly predicting outputs from context, the architecture separates state-space navigation from neural inference. Neural components infer target directions within a vector space, while graph-constrained decoding determines admissible state transitions. The resulting design prioritizes explicit state representation, traceability, controllability, and inspection over unrestricted emergence.

Importantly, VGPC is not intended as a replacement for transformer-based systems. Rather, both approaches are viewed as complementary components within hybrid intelligent systems.

### 2.3 Neuro-Symbolic Artificial Intelligence

A long-standing objective in artificial intelligence has been the integration of neural learning and symbolic reasoning. Neuro-symbolic systems attempt to combine the adaptability of neural networks with the interpretability and structure of symbolic representations.

The VGPC architecture can be viewed as a neuro-symbolic framework in this broader sense. Neural inference is responsible for estimating future state directions, while symbolic and deterministic mechanisms such as graphs, transition rules, and evaluation procedures constrain and guide system behavior. Unlike many neuro-symbolic approaches that focus on logical reasoning, VGPC is centered around structured state-space traversal and hierarchical generation processes.

### 2.4 Graph-Based Models

Graphs have been applied extensively throughout artificial intelligence, including knowledge graphs, probabilistic graphical models, planning systems, state-space search algorithms, and graph neural networks.

The role of graphs in VGPC differs from these applications. The graph does not primarily represent semantic knowledge or relational facts. Instead, it encodes admissible transitions between system states. In this sense, the graph functions as an explicit representation of the state space that governs possible system trajectories.

This design shares certain similarities with search-based planning systems and state-transition models, while maintaining compatibility with learned vector representations and neural guidance mechanisms.

### 2.5 Reflection and Iterative Improvement Mechanisms

Recent AI systems increasingly employ self-reflection, verification, critique, and iterative refinement processes. Examples include chain-of-thought verification, self-consistency methods, planning agents, and various forms of generate-and-evaluate workflows.

The PRE (Propose–Retry–Evaluate) mechanism used within VGPC belongs to this general family of approaches. However, PRE is incorporated as an architectural component rather than merely a prompting strategy. It can operate at multiple levels of abstraction, allowing recursive evaluation and improvement of state sequences, plans, actions, or generated structures.

### 2.6 Positioning of VGPC

The VGPC architecture family intersects multiple existing directions but does not fully coincide with any of them. It combines:

- vector-based representations inspired by modern neural systems,
- explicit state transitions inspired by planning and symbolic AI,
- graph-constrained state spaces,
- recursive evaluation through PRE mechanisms,

- behavioral steering through CAS vectors,
- optional rule-based control and deterministic reasoning.

The resulting architecture is intended as a framework for constructing inspectable, controllable, and modular intelligent systems. Within the Triadic Cosmos ecosystem, the previously introduced GLP–Graph architecture represents an early concrete instance of this broader architectural family.

# 3 Design Objectives

The Vector–Graph–PRE–CAS (VGPC) Cognitive Architecture Family was designed to address several objectives that emerge from the development of hybrid intelligent systems. Rather than focusing exclusively on a particular domain, such as language generation, the architecture targets a broader class of problems that can be represented as state spaces with explicit transitions between states.

The central assumption of the framework is that many cognitive, generative, and decision-making processes can be modeled as navigation through a structured state space. A state may represent a token, a sentence, a paragraph, an image component, an audio fragment, a world configuration, a memory item, an action, or any other domain-specific unit. Intelligent behavior then emerges from the selection and evaluation of transitions between valid states.

The design of the architecture is guided by five primary objectives.

## 3.1 Modularity

The architecture is intended to be composed of independent but cooperating subsystems. State representation, graph traversal, neural inference, behavioral steering, evaluation, and rule-based control are separated into distinct architectural components.

This modularity allows systems to be extended, modified, or replaced without requiring the complete redesign of the overall architecture. It also supports the construction of hybrid systems in which different cognitive functions are implemented using different computational mechanisms.

## 3.2 Traceability

Modern neural systems often achieve powerful results at the cost of reduced interpretability. The VGPC architecture seeks to preserve explicit knowledge about how decisions are made.

The use of graph-structured state spaces, deterministic transitions, explicit decoding mechanisms, and optional rule-based constraints makes it possible to inspect how a particular output or decision was produced. The resulting systems therefore provide significantly greater transparency than purely end-to-end neural architectures.

## 3.3 Controllability

The architecture is designed to permit explicit control over system behavior without requiring extensive prompt engineering or retraining.

Behavior can be influenced through graph structure, rule systems, modulation mechanisms, evaluation loops, and CAS vectors. Multiple layers of control can therefore coexist within the same system, allowing behavioral constraints to be applied at different stages of the inference process.

This objective is particularly important for systems operating in environments where consistency, safety, reliability, or policy compliance must be maintained.

### 3.4 Hybrid Intelligence

The architecture follows a Hybrid Intelligence philosophy in which intelligence is viewed as the interaction of multiple complementary mechanisms rather than the product of a single algorithmic principle.

Neural inference, symbolic constraints, deterministic graphs, recursive evaluation mechanisms, and behavioral steering vectors are all treated as legitimate cognitive components. The architecture is therefore intended to serve both as a standalone framework and as a building block within larger hybrid intelligent systems.

In particular, VGPC systems are designed to coexist naturally with large language models, planning systems, memory systems, and specialized reasoning engines.

### 3.5 Domain Independence

Although some concrete implementations of the architecture have focused on text generation, the VGPC framework is not inherently tied to language.

The architecture is intended for any domain that can be represented as states and transitions between states. Examples include text generation, image generation, audio generation, simulation systems, planning systems, memory systems, game worlds, autonomous agents, and other structured cognitive processes.

The architecture therefore defines a general computational framework rather than a domain-specific generative model. Text generation, including systems such as GLP–Graph, represents only one possible application of the broader architectural family.

### 3.6 Structured Emergence

The architecture seeks to combine the flexibility of learned representations with the reliability of explicit structural constraints.

Rather than maximizing unrestricted emergence, VGPC promotes emergence within well-defined state spaces. Neural components may discover useful transition patterns and representations, while graphs, rules, and evaluation mechanisms ensure that generated trajectories remain compatible with the constraints of the modeled domain.

The objective is not to eliminate emergence, but to place it within a traceable, inspectable, and controllable computational framework.

## 4 The Vector–Graph–PRE–CAS Cognitive Architecture Family

The Vector–Graph–PRE–CAS (VGPC) Cognitive Architecture Family provides a general framework for constructing intelligent systems based on explicit state spaces, learned transition prediction, behavioral control mechanisms, and recursive evaluation processes. Rather than being tied to a particular domain or representation format, the architecture is intended for any problem that can be modeled as a collection of states and admissible transitions between those states.

At its core, the architecture combines six complementary concepts:

1. Vector-based state representations.
2. Graph-defined transition spaces.
3. Neural state-vector inference.
4. Optional vector modulation mechanisms.
5. PRE-based recursive evaluation loops.
6. CAS-based behavioral steering.

The interaction of these components enables the construction of systems that are simultaneously trainable, controlled, traceable, and modular. The resulting architecture occupies an intermediate position between purely neural systems and purely symbolic systems by combining learned representations with explicit state-space structure.

Unlike architectures built around direct output prediction, VGPC models cognition as structured navigation through a state space. Neural components are responsible for estimating desirable future regions of that state space, while graphs, rules, and evaluation mechanisms constrain how movement through that space ultimately occurs.

## 4.1 Architectural Overview

The VGPC architecture assumes the existence of a state space containing all possible states relevant to a given domain. Each state is represented by an associated vector and may correspond to any meaningful unit of representation.

Examples include:

- tokens, sentences, paragraphs, or chapters in text generation,
- image primitives, regions, objects, or latent image states in visual systems,
- audio fragments, phonetic units, or musical structures in audio systems,
- actions and world states in planning systems,
- memory objects in memory architectures,
- entities, concepts, or beliefs in reasoning systems.

A graph defines which transitions between states are admissible. This graph may be derived from training data, manually specified, partially learned, or constructed through domain-specific mechanisms.

The current context is encoded into a context representation that captures the relevant system state. A neural inference mechanism predicts a target state vector representing the desired direction of the next transition. Optional modulation mechanisms may then transform this vector before decoding occurs.

The graph is used to enumerate candidate states reachable from the current state, after which decoding mechanisms determine which candidate most closely matches the inferred target direction. Additional rule systems may further filter or constrain candidate transitions.

The resulting transition can subsequently be evaluated through one or more PRE (Propose–Retry–Evaluate) loops. If accepted, the new state becomes part of the active context. Otherwise, alternative candidates may be explored.

Conceptually the architecture can be summarized as:

$$\text{Context} \rightarrow \text{Vector Inference} \rightarrow \text{Vector Modulation} \rightarrow \text{Graph-Constrained Decoding} \rightarrow \text{PRE Evaluation} \rightarrow \text{Context Update}$$

## 4.2 Inference Pipeline

The VGPC inference process can be viewed as a sequence of distinct cognitive operations.

First, the current context is encoded. The exact encoding mechanism is domain-dependent and may incorporate recent states, historical information, entity tracking, memory structures, positional information, behavioral control vectors, or additional contextual information.

Second, a neural inference mechanism predicts a target state vector. This vector does not necessarily correspond to an actual state. Instead, it represents a desired direction within the state space toward which the system should move.

Third, optional modulation mechanisms may alter the inferred vector. These modulations may depend on contextual variables, positional information, CAS vectors, domain-specific constraints, or learned control mechanisms.

Fourth, decoding is performed. The graph provides a set of admissible transition candidates. Candidate state vectors are compared with the inferred target vector using an appropriate distance measure, such as cosine similarity or Euclidean distance. The resulting scores are transformed into a probability distribution or ranking from which candidate states can be selected.

Fifth, one or more PRE evaluation stages may assess the quality of the generated transition or sequence. Evaluation can occur at multiple levels of abstraction, including individual states, state sequences, plans, documents, images, actions, or larger cognitive structures.

Finally, the accepted state is incorporated into the active context, and the process repeats.

The architecture therefore separates:

- **Inference**: determining where to move.
- **Decoding**: determining what is possible.
- **Evaluation**: determining what is acceptable.

This separation allows different computational mechanisms to specialize in different aspects of cognition.

### 4.3 Architectural Variants

The VGPC framework describes a family of architectures rather than a single fixed design. Different implementations may vary in how states are defined, how context is represented, and how inference, control, and evaluation are performed.

A first distinction concerns the granularity of states. Fine-grained systems may operate on small units such as tokens, actions, or image fragments, while coarser systems may operate on sentences, plans, scenes, objects, or higher level abstractions.

A second distinction concerns hierarchy. Multiple VGPC instances may cooperate across levels of abstraction. Higher-level systems can infer transitions between abstract states while lower-level systems realize these transitions through finer-grained state generation processes.

A third distinction concerns the role of PRE evaluation. Some implementations may perform evaluation only after complete sequences have been generated, whereas others may perform recursive evaluation at every stage of generation.

A fourth distinction concerns behavioral control. CAS vectors may be absent, fixed, dynamically adjusted, or learned through domain-specific training procedures. Rule systems may similarly range from completely absent to highly restrictive deterministic policies.

Finally, VGPC systems may operate either as standalone cognitive systems or as components within larger hybrid architectures. In hybrid environments, a VGPC instance may serve as a structured world engine, planning subsystem, memory subsystem, reasoning component, or behavioral controller while cooperating with other specialized modules such as large language models, retrieval systems, external memory stores, or agentic orchestration layers.

Consequently, the VGPC framework should not be viewed as a single architecture but rather as a structured design space for constructing modular and controllable intelligent systems across a wide range of domains.

## 5 State Representation

### 5.1 States as Vectors

The fundamental building block of the Vector–Graph–PRE–CAS (VGPC) architecture is the notion of a *state*. All cognitive processes described by the framework are modeled as transitions between states

within a state space.

A state represents a meaningful unit within a domain. The exact interpretation of a state is domain-dependent and may range from low-level elements to highly abstract constructs.

Examples include:

- tokens, words, sentences, paragraphs, or chapters in text systems,
- image primitives, objects, or regions in visual systems,
- audio fragments, phonemes, or musical structures in audio systems,
- actions or plans in autonomous agents,
- memories, concepts, or beliefs in cognitive systems,
- world configurations in simulation environments.

Each state is associated with a vector representation.

$$s_i \rightarrow \mathbf{v}_i$$

where $s_i$ denotes a state and $\mathbf{v}_i$ denotes the corresponding state vector.

The state vector serves as the primary representation used by neural inference, distance calculations, steering mechanisms, and candidate selection algorithms. The vector may be deterministic, learned from data, procedurally generated, or produced through a combination of these approaches.

The architecture does not prescribe a particular vector construction method. Any representation capable of capturing useful similarities and transitions between states is compatible with the framework.

## 5.2 Finite and Procedural State Spaces

Many practical implementations employ a finite state space.

For example, in a text generation system a state may correspond to a token from a finite vocabulary. In such systems every admissible state is known in advance and can be explicitly represented within the graph structure.

However, finite state spaces are not a requirement of the VGPC architecture.

More generally, a state space may be partially procedural or entirely procedural. In such systems states are generated dynamically rather than being enumerated beforehand.

The architecture only requires that for a current state:

1. a vector representation can be obtained,
2. admissible successor states can be determined,
3. successor states can also be represented as vectors.

Consequently, the graph itself may be explicit, implicit, or procedural.

An explicit graph stores transitions directly as nodes and edges.

A procedural graph computes transitions through algorithms, simulation rules, constraint systems, generators, or domain-specific logic.

From the perspective of the architecture these approaches are equivalent, provided that candidate transitions can be generated when required.

### 5.3 Hierarchical State Spaces

States may exist at multiple levels of abstraction simultaneously.

For example, a text generation system may represent:

$$\text{Book} \rightarrow \text{Chapter} \rightarrow \text{Paragraph} \rightarrow \text{Sentence} \rightarrow \text{Token}$$

where each level consists of its own states, transitions, and vector representations.

Similarly, a visual generation system may contain hierarchical states ranging from complete scenes to objects, regions, shapes, and image primitives.

The VGPC framework places no restrictions on the granularity of states. Different implementations may operate at a single level of abstraction or may combine multiple interconnected state spaces.

Hierarchical state spaces enable higher-level cognitive processes to guide the behavior of lower-level generation processes while preserving explicit state representations throughout the system.

### 5.4 Context Representation

A single state rarely contains sufficient information to determine future behavior. Consequently, VGPC systems generally operate on a context consisting of multiple states.

The current context can be interpreted as the active cognitive state of the system.

$$C = \{s_1, s_2, ..., s_n\}$$

The method used to encode this context is implementation-dependent and may include recent states, long-term memories, entity tracking information, positional representations, behavioral steering vectors, or other domain-specific features.

The resulting context representation forms the input to the neural inference mechanisms described in subsequent sections.

In this sense, the context plays a role analogous to the query representation used in modern neural architectures, while the state space and associated transitions define the structured environment within which inference occurs.

## 6 Graph-Based State Spaces

A distinguishing characteristic of the Vector–Graph–PRE–CAS (VGPC) architecture family is the explicit representation of admissible transitions between states. While vector representations capture similarity, direction, and learned relationships between states, the graph structure defines the boundaries within which state transitions may occur.

The graph therefore acts as a structured state space that constrains possible system trajectories. Rather than allowing arbitrary movement within a vector space, cognition is modeled as navigation through a graph of admissible transitions guided by learned vector inference mechanisms.

This separation between vector prediction and graph traversal forms one of the central design principles of the architecture.

### 6.1 Transition Graphs

Let

$$G = (S, E)$$

denote a directed graph where $S$ represents the set of states and $E$ represents the set of admissible transitions between states.

An edge

$$(s_i, s_j) \in E$$

indicates that a transition from state $s_i$ to state $s_j$ is permitted by the system.

Unlike many graph-based AI systems, the graph in VGPC is not primarily intended to represent semantic knowledge, logical facts, or entity relationships. Instead, the graph defines the possible evolution of system states.

The graph can be constructed through multiple mechanisms:

- learned directly from training data,
- manually engineered,
- generated procedurally,
- derived from simulations,
- generated dynamically during execution,
- constructed as a combination of these approaches.

For example, in a language generation system, transitions may represent observed continuations between language units. In a planning environment, transitions may correspond to valid actions that transform one world state into another. In a visual system, transitions may describe admissible modifications of a scene representation.

The architecture deliberately avoids imposing restrictions on how transitions are obtained. The only requirement is that a mechanism exists for determining which successor states are reachable from a given state.

Consequently, the graph may be:

- explicit and finite,
- implicit and generated on demand,
- partially procedural,
- potentially unbounded.

This distinction is important because the VGPC architecture does not require a fixed vocabulary or a finite state space. Infinite or dynamically generated state spaces are compatible with the framework provided that admissible transitions can be generated when required.

## 6.2 Candidate State Generation

During inference the graph functions as a candidate generator.

Given the current state $s_t$, the graph determines the set of admissible successor states:

$$N(s_t) = \{s_j \mid (s_t, s_j) \in E\}$$

where $N(s_t)$ denotes the neighborhood of the current state.

This neighborhood constitutes the candidate set from which future states may be selected.

In explicit graphs, candidate generation consists simply of enumerating the outgoing transitions of the current state.

In procedural graphs, candidate generation may require the execution of algorithms, simulations, rule systems, constraint solvers, environment models, or domain-specific generators.

The architecture intentionally separates candidate generation from candidate selection.

Candidate generation answers the question:

*What transitions are possible?*

Candidate selection answers the question:

*Which transition should be preferred?*

The first question is answered by the graph.

The second question is answered by the vector inference and decoding mechanisms described in subsequent sections.

This separation introduces a clear distinction between the structure of the state space and the learned preferences of the system.

### 6.3 Graph Traversal as Inference

In many contemporary neural architectures, inference directly predicts the next output from the current context.

The VGPC architecture adopts a different perspective.

Inference is not viewed as direct output prediction but rather as guided navigation through a constrained state space.

Given a current context, the neural inference system predicts a target vector representing a desirable future direction within the state space. The graph then determines which states are actually reachable.

The resulting inference process can therefore be conceptualized as:

$$\text{Context} \rightarrow \text{Target Direction} \rightarrow \text{Reachable Candidates} \rightarrow \text{Candidate Selection}$$

rather than:

$$\text{Context} \rightarrow \text{Output}$$

This distinction has important consequences.

First, all generated states remain compatible with the graph structure and any additional constraints imposed by the system.

Second, reasoning becomes inherently traceable because every generated result corresponds to an explicit sequence of graph transitions.

Third, graph traversal naturally supports deterministic, probabilistic, neural, or hybrid decision mechanisms without requiring changes to the underlying state space representation.

The graph can therefore be interpreted as a structured environment through which cognition occurs. Neural inference mechanisms do not directly determine future states. Instead, they guide traversal within the space of admissible possibilities provided by the graph.

This design allows learned behavior and explicit structure to coexist within the same cognitive framework, balancing adaptability with interpretability and control.

## 7 Neural State Vector Inference

While the graph defines the admissible structure of the state space, the role of neural inference is to determine which directions within that state space are desirable given the current context.

The purpose of neural inference is therefore not to directly select the next state, but to estimate a target state vector that represents a suitable direction for future state transitions. The graph, candidate generation mechanisms, and decoding procedures subsequently determine which concrete state is chosen.

This separation between direction prediction and state selection constitutes a central design principle of the VGPC architecture family.

## 7.1 Context Encoding

The first stage of neural inference consists of constructing a context representation from the current cognitive state of the system.

Unlike architectures that prescribe a specific context encoding mechanism, the VGPC framework deliberately remains agnostic regarding how context is represented. Different domains, applications, and cognitive functions may benefit from different encoding strategies.

Consequently, multiple context encoders may coexist within the same system, with each encoder capturing information at a different resolution or serving a different purpose.

Possible context components include:

- recent state histories,
- positional information,
- long-term memory representations,
- entity tracking information,
- world-state representations,
- planning information,
- temporal information,
- external sensor inputs,
- CAS vectors,
- domain-specific features.

These components can be concatenated, combined through learned projections, fused through attention mechanisms, or integrated using any other suitable encoding approach.

Importantly, the architecture does not assume a single universal encoding paradigm. Self-attention mechanisms may be used when appropriate, but they represent only one possible solution among many.

For example, some implementations may rely primarily on:

- attention-based context aggregation,
- recurrent encoders,
- memory retrieval systems,
- statistical summaries,
- explicit state tracking,
- graph-derived features,
- procedural feature generators.

The result of this stage is an input vector

$$\mathbf{x}$$

that captures the information considered relevant for future state prediction.

From the perspective of the architecture, the construction of this context vector is a design choice rather than a fixed requirement.

## 7.2 Target State Vector Prediction

The encoded context vector serves as input to a neural prediction mechanism whose purpose is to infer a target state vector.

$$\mathbf{x} \rightarrow f_\theta \rightarrow \hat{\mathbf{v}}$$

where:

- $\mathbf{x}$ is the encoded context vector,
- $f_\theta$ is the inference model,
- $\hat{\mathbf{v}}$ is the predicted target state vector.

The target vector does not necessarily correspond to an existing state.

Instead, it represents a desired direction within the state space and acts as a guiding signal for subsequent decoding mechanisms.

The VGPC architecture deliberately imposes few restrictions on the structure of the inference model.

Simple implementations may use shallow multilayer perceptrons, while more advanced systems may employ:

- deep feed-forward networks,
- recurrent architectures,
- gated architectures,
- recursive neural systems,
- attention-based components,
- mixture-of-experts systems,
- dynamically generated subnetworks,
- hypernetwork-generated parameters,
- other differentiable inference mechanisms.

The architecture only requires that a mapping exists between the encoded context and a predicted target vector.

Consequently, VGPC defines the role of the inference mechanism rather than a specific neural architecture.

This design allows future implementations to adopt new neural techniques without altering the overall architecture.

## 7.3 Learning Transition Directions

Training the inference mechanism consists of learning a mapping between observed contexts and desirable future state vectors.

Given a context representation

$$\mathbf{x}$$

and an observed successor state

$$s_{t+1}$$

with associated vector

$$\mathbf{v}_{t+1},$$

the objective is to train the inference model such that

$$f_\theta(\mathbf{x}) \approx \mathbf{v}_{t+1}.$$

The resulting loss function depends on the chosen vector representation and application domain. Common objectives may include:

- cosine similarity losses,
- Euclidean distance losses,
- contrastive objectives,
- ranking losses,
- learned metric-space objectives,
- composite multi-objective losses.

From a conceptual perspective, the training objective is straightforward:

*Given the current context, learn the direction in state space that leads toward appropriate future states.*

This differs from direct next-state prediction because the neural model is not required to identify the exact successor state. Instead, it learns the underlying geometric structure of transitions within the state space.

During inference, graph traversal and decoding mechanisms use this predicted direction to select admissible successor states.

The resulting division of responsibilities can be summarized as follows:

- Context encoding determines what information is relevant.
- Neural inference predicts where the system should move.
- Graph traversal determines what transitions are possible.
- Decoding determines which admissible transition best matches the predicted direction.

This separation allows neural learning, symbolic constraints, procedural knowledge, and deterministic control mechanisms to coexist within a unified cognitive architecture while preserving traceability and modularity.

# 8 Vector Modulation

The target state vector produced by the neural inference stage represents an estimate of the direction that should be followed within the state space. However, there is no requirement that this vector must be used directly during decoding.

The VGPC architecture introduces an optional *vector modulation* stage between neural inference and graph-based decoding. The purpose of this stage is to transform, refine, or adapt the predicted target vector before candidate selection occurs.

Conceptually, vector modulation acts as an intermediate control layer situated between learned prediction and state-space traversal.

$$\mathbf{x} \rightarrow f_\theta \rightarrow \hat{\mathbf{v}} \rightarrow M \rightarrow \hat{\mathbf{v}}'$$

where:

- $\mathbf{x}$ is the encoded context,
- $\hat{\mathbf{v}}$ is the inferred target vector,
- $M$ is a modulation function,
- $\hat{\mathbf{v}}'$ is the modulated target vector.

The modulated vector is subsequently used during candidate evaluation and decoding.

This additional layer increases the flexibility of the architecture by allowing systematic transformations to be learned independently from the primary inference process.

## 8.1 Motivation

In many learning systems, a single neural inference mechanism is expected to capture every aspect of future state prediction.

The VGPC architecture adopts a different perspective.

A predicted target vector may correctly identify a desired region of the state space while still benefiting from additional transformations that improve decoding performance, reinforce domain structure, or inject behavioral control.

The motivation for vector modulation is therefore to introduce an additional adaptive layer capable of modifying target directions without requiring changes to the underlying inference model.

Several potential advantages arise from this separation:

- refinement of inferred target directions,
- incorporation of domain-specific structure,
- contextual adaptation of predictions,
- behavioral steering,
- improved decoding performance,
- selective specialization of state transitions.

Unlike large neural components that may require extensive retraining, a modulation layer can often be learned independently while leaving the underlying inference mechanism unchanged.

Within the GLP–Graph architecture, preliminary experiments have demonstrated that learned vector modulation can have a measurable influence on generation behavior despite its conceptual simplicity. This suggests that target-vector transformation may represent a useful source of additional expressiveness within the broader VGPC design space.

## 8.2 Projection Functions

The VGPC framework does not prescribe a particular modulation mechanism.

The simplest implementation consists of a projection matrix:

$$\hat{\mathbf{v}}' = W\hat{\mathbf{v}}$$

where $W$ is a learnable square matrix with the same dimensionality as the state vectors.

Such a projection can be interpreted as a learned transformation of the state space that alters how inferred directions are mapped into candidate-selection space.

More generally, modulation functions may include:

- linear projections,

- affine transformations,
- multiple projection matrices,
- learned neural transformations,
- recursive modulation systems,
- procedural transformations,
- hybrid deterministic-neural mechanisms.

The projection mechanism itself may be globally applied to all states or may be specialized to particular contexts, state categories, abstraction levels, or behavioral modes.

Importantly, useful modulation does not necessarily require complex models.

Simple matrix-based projection layers may already provide meaningful improvements while remaining fully interpretable and computationally efficient.

Furthermore, the architecture permits modulation layers to be trained independently from the primary inference network. They may therefore function as lightweight adaptation mechanisms rather than large-scale learned models.

### 8.3 Conditional Modulation

Modulation functions may also depend on contextual information.

Rather than applying a single global transformation, different modulation functions may be activated under different conditions.

For example, modulation may depend on:

- state position,
- generation stage,
- hierarchy level,
- domain context,
- task identity,
- system mode,
- CAS vectors,
- external control signals.

In such systems the modulation process becomes:

$$\hat{\mathbf{v}}' = M(\hat{\mathbf{v}}, \mathbf{c})$$

where $\mathbf{c}$ represents conditional control information.

A particularly important possibility is conditioning modulation on CAS vectors. Different content, accuracy, safety, or personality profiles may induce different transformations of the same inferred direction. In this way, behavioral steering can occur not only at the context level but also within the geometry of the state-space traversal process itself.

The mechanism generating the modulation parameters may also vary.

Projection matrices may be:

- fixed after training,

- selected from a predefined set,
- interpolated between multiple matrices,
- generated by auxiliary neural systems,
- procedurally constructed,
- produced by hypernetworks.

Likewise, the learning process itself is not restricted. Standard gradient optimization may be used, but alternative approaches remain equally compatible.

For example, matrix-based projection layers may be learned incrementally through local update mechanisms inspired by Hebbian learning, where individual matrix coefficients evolve according to observed transition patterns. Consequently, modulation can function both as a conventional trainable layer and as an independently adaptable control mechanism.

The VGPC framework therefore treats vector modulation not as a fixed algorithm but as a general architectural concept: an optional transformation stage that reshapes predicted state-space directions before graph traversal and decoding occur. This additional degree of freedom enables learned adaptation and behavioral control while preserving the modular design principles of the architecture.

# 9 Decoding Mechanisms

The purpose of decoding is to transform a predicted target vector into a concrete state transition.

Within the VGPC architecture, decoding operates on the candidate states provided by the graph and any additional filtering mechanisms that may be present. Unlike direct prediction architectures, decoding is not responsible for generating arbitrary outputs. Instead, it selects among admissible alternatives that already exist within the state space.

This separation between candidate generation and candidate selection is one of the defining characteristics of the architecture.

$$\text{Context} \rightarrow \text{Target Vector} \rightarrow \text{Candidate Generation} \rightarrow \text{Decoding} \rightarrow \text{Selected State}$$

The framework intentionally imposes few restrictions on the decoding mechanism. Any procedure that transforms a set of admissible candidate states into a selected successor state is compatible with the architecture.

## 9.1 Candidate Selection

The first stage of decoding consists of obtaining a candidate set.

Given a current state $s_t$, the graph produces a set of admissible successor states

$$N(s_t) = \{s_1, s_2, \ldots, s_n\}.$$

Additional mechanisms may further reduce this candidate set before selection occurs.

Possible filtering mechanisms include:

- deterministic rules,
- constraint systems,
- safety policies,
- CAS-dependent restrictions,
- planning constraints,

- procedural validators,
- domain-specific logic.

The resulting candidate set therefore represents the states that are both reachable and currently acceptable.

This process answers the question:

*Which transitions are permitted?*

Only after this stage does the architecture address the question:

*Which permitted transition should be chosen?*

## 9.2 Distance Metrics

Once the candidate set has been established, each candidate state vector is compared to the predicted target vector.

Let

$$\hat{\mathbf{v}}$$

denote the inferred or modulated target vector and

$$\mathbf{v}_i$$

the vector associated with candidate state $s_i$.

A scoring function

$$score(s_i) = D(\hat{\mathbf{v}}, \mathbf{v}_i)$$

can then be applied to determine how well a candidate aligns with the predicted direction.

The architecture does not require a specific distance metric.

Possible choices include:

- cosine similarity,
- Euclidean distance,
- Manhattan distance,
- learned similarity metrics,
- kernel-based similarity measures,
- composite scoring functions.

Vector representations make such comparisons straightforward while remaining independent of the underlying domain.

In many practical implementations cosine similarity is particularly attractive because it primarily evaluates directional agreement rather than absolute magnitude.

However, the VGPC framework intentionally leaves metric selection to the application.

## 9.3 Probability Assignment

The candidate scores obtained during distance evaluation must ultimately guide candidate selection.

Unlike large language models, the resulting probability values should not necessarily be interpreted as statistical estimates of token frequencies or next-state likelihoods. Instead, they serve as a prioritization mechanism that expresses the relative desirability of candidate transitions.

Consequently, the VGPC framework does not prescribe a particular probability assignment procedure.

A variety of approaches are possible:

- direct score normalization,
- softmax-based distributions,
- rank-based probability assignment,
- boosted ranking systems,
- deterministic best-candidate selection,
- uniform sampling among high-ranking candidates.

In practice, softmax is not always ideal because small score differences can sometimes produce excessively peaked distributions.

Alternative normalization procedures may therefore be preferable.

For example, a simple normalization scheme consists of:

$$score'_i = score_i - \min(score) + T$$

where $T$ is a temperature parameter ensuring positive values.

The resulting probabilities become:

$$P(s_i) = \frac{score'_i}{\sum_j score'_j}.$$

This approach preserves relative ranking while allowing the temperature parameter to control diversity.

Larger temperature values tend to distribute probability mass more evenly, while smaller values increase preference for the highest-scoring candidates.

Additional mechanisms may further modify candidate priorities before normalization occurs.

For example, the highest-ranked candidates may receive an explicit score boost prior to probability assignment, thereby increasing their likelihood of selection without completely eliminating lower-ranked alternatives.

The architecture remains agnostic regarding which prioritization strategy is used.

## 9.4 Top-K Selection

In many systems the full candidate set may be too large to evaluate exhaustively during final selection.

A common strategy therefore consists of retaining only the highest-scoring candidates:

$$K = \{s_1, s_2, \ldots, s_k\}.$$

The remaining candidates are discarded and subsequent probability assignment or sampling procedures operate only on this reduced set.

Top-K selection offers several advantages:

- reduced computational cost,

- improved selection efficiency,
- removal of low-quality candidates,
- controllable diversity.

The architecture does not mandate a particular strategy after the Top-K set has been determined. Possible approaches include:

- selecting the highest-ranked candidate,
- probability-weighted sampling,
- equal-probability sampling,
- CAS-conditioned sampling,
- rule-guided selection,
- evaluator-guided selection.

Consequently, Top-K should be viewed as a candidate reduction mechanism rather than a complete decoding strategy.

Within the VGPC architecture, decoding is ultimately responsible for transforming geometric preferences within vector space into concrete state transitions. By separating graph traversal, candidate generation, score evaluation, probability assignment, and final selection, the framework allows deterministic, probabilistic, neural, symbolic, and hybrid decision mechanisms to coexist within a unified state-space representation.

# 10 The CAS Framework

The Content–Accuracy–Safety–Personality (CAS) framework provides a general mechanism for behavioral steering within the VGPC architecture family.

Unlike rule systems, which impose explicit constraints on admissible behavior, CAS operates as a soft conditioning mechanism. Its purpose is not to determine what the system must or must not do, but rather to influence the probability that particular behaviors, trajectories, styles, or decisions emerge during generation.

The central idea is that training data can be annotated with behavioral metadata that captures desirable properties of states, sequences, or complete artifacts. This metadata is represented as a vector and incorporated into both training and inference.

More generally, the CAS framework can be interpreted as a mechanism for encoding additional dimensions of behavior alongside the primary domain representation.

$$CAS = (c_1, c_2, \ldots, c_n)$$

where each component represents a continuous behavioral characteristic.

Although the original formulation consists of Content, Accuracy, Safety, and Personality dimensions, the VGPC architecture imposes no restrictions on the number or meaning of components.

CAS should therefore be viewed as a general behavioral vector framework rather than a fixed four-dimensional representation.

## 10.1 Content

The Content dimension captures what type of information, themes, concepts, or subject matter are associated with a particular state or sequence.

Depending on the application domain, content-related dimensions may encode:

- topics,
- genres,
- semantic categories,
- conceptual domains,
- world-specific attributes,
- stylistic themes.

In generative systems this allows behavioral steering toward particular forms of content without explicitly prescribing individual outcomes.

For example, two trajectories may remain structurally valid while differing substantially in their thematic tendencies.

The Content dimensions provide a mechanism for expressing such preferences within the architecture.

## 10.2 Accuracy

The Accuracy dimension captures properties related to correctness, reliability, consistency, or quality.

The precise interpretation depends on the domain.

Possible examples include:

- factual correctness,
- simulation fidelity,
- logical consistency,
- structural coherence,
- domain compliance,
- adherence to reference data.

Accuracy does not directly enforce correctness.

Instead, it biases generation toward states and transitions that have been associated with higher levels of the desired property during training.

Explicit guarantees remain the responsibility of graph constraints, rule systems, validation components, or PRE evaluation loops.

## 10.3 Safety

The Safety dimension captures properties associated with acceptable and responsible system behavior.

Examples may include:

- risk levels,
- content sensitivities,

- operational constraints,
- policy preferences,
- domain-specific safety criteria.

The role of CAS safety differs fundamentally from deterministic safety systems.

A high safety value encourages trajectories that resemble safe examples present within the training data.

However, it does not prevent unsafe transitions from occurring if they remain reachable and admissible.

Consequently, safety-critical guarantees should generally be implemented through rule-based constraints or evaluation mechanisms rather than CAS alone.

## 10.4 Personality

The Personality dimension captures stable behavioral tendencies that influence how the system behaves while preserving freedom within the underlying state space.

Possible examples include:

- creativity,
- formality,
- exploration,
- curiosity,
- cautiousness,
- narrative style,
- agent identity.

Personality dimensions are particularly useful because they allow behavioral characteristics to be learned directly from examples rather than manually encoded through rules.

Multiple personalities may coexist within the same dataset and become available through continuous CAS interpolation.

As a result, personality becomes a steerable property of generation rather than a fixed characteristic of the system.

## 10.5 Behavioral Steering Through CAS Vectors

The primary purpose of CAS is behavioral steering.

During training, examples are associated with one or more CAS vectors. The neural inference mechanisms therefore learn not only the relationship between context and future states, but also the influence of behavioral attributes on those transitions.

During inference, a desired CAS vector can be incorporated into the context representation:

$$\mathbf{x} = Context \oplus CAS$$

where $\oplus$ denotes some form of feature combination.

The inference model then predicts transition directions that are influenced by both the contextual state and the requested behavioral profile.

Importantly, CAS should not be interpreted as a rule system.

A common misconception is to view CAS as a form of whitelisting or blacklisting. Such mechanisms are outside its intended purpose.

The objective of CAS is not:

- to forbid states,
- to enforce deterministic policies,
- to guarantee compliance,
- to replace symbolic constraints.

Instead, CAS acts as a soft biasing mechanism.

It influences preferences without eliminating alternatives.

For example, a high safety value may increase the probability of selecting safer trajectories, but it does not automatically remove all unsafe possibilities from the candidate set.

Similarly, a creativity-oriented personality vector may increase the likelihood of unusual transitions without explicitly requiring them.

The distinction between CAS and rule systems is therefore fundamental:

- Rules determine what is permitted.
- Graphs determine what is possible.
- CAS influences what is preferred.

Because CAS operates through continuous vector representations, heterogeneous datasets can be annotated with rich behavioral information that remains available during inference. The resulting mechanism provides a flexible and domain-independent approach to steering behavior while preserving the adaptability and diversity of the underlying state-space traversal process.

Within the VGPC architecture family, CAS therefore serves as a bridge between explicit control and learned behavior, enabling systems to express preferences without sacrificing flexibility or requiring hard-coded policies.

# 11 Rule-Based Control

Although graph-constrained state spaces already provide a significant degree of structural control, additional rule-based mechanisms may further restrict or guide system behavior.

The VGPC architecture therefore includes optional rule-based control systems that operate alongside neural inference and graph traversal. These mechanisms allow deterministic knowledge, domain constraints, policies, and symbolic reasoning procedures to directly influence state transitions.

The architecture does not require rule systems, and many applications may operate entirely through graph constraints and vector-guided decoding. Nevertheless, rule-based control can provide additional guarantees regarding consistency, safety, validity, and domain-specific correctness.

Within the broader architecture, rules represent an additional source of knowledge that complements both learned representations and graph structure.

## 11.1 Transition Constraints

The most direct application of rule-based control is the restriction of candidate transitions.

Given a candidate set

$$N(s_t) = \{s_1, s_2, \ldots, s_n\},$$

a rule system may remove candidates that violate predefined constraints.

Conceptually:

$$N(s_t) \to R(N(s_t)) \to N'(s_t)$$

where $R$ denotes the rule system and $N'(s_t)$ the filtered candidate set.
Possible constraints include:

- syntactic validity,
- structural consistency,
- domain-specific policies,
- safety requirements,
- resource limitations,
- planning constraints,
- behavioral restrictions.

Rules may operate on:

- the current state,
- candidate states,
- state histories,
- contextual information,
- CAS vectors,
- external environment information.

In practice, such constraints often eliminate undesirable transitions before candidate selection occurs, reducing the burden placed on neural inference and evaluation mechanisms.

Importantly, graph constraints and rule constraints serve different purposes.

The graph defines transitions that are generally admissible within the modeled domain.

Rules determine which of those transitions are currently acceptable under the present context, objectives, or policies.

Consequently, a transition may exist within the graph while still being temporarily disallowed by the active rule system.

## 11.2 Deterministic Reasoning

Beyond candidate filtering, rule systems may perform deterministic reasoning directly.

In this setting, rules are not merely restrictions but active decision-making mechanisms.

Examples include:

- enforcing domain invariants,
- performing logical deductions,
- managing state consistency,
- applying planning heuristics,
- maintaining structural constraints,
- triggering mandatory transitions.

In certain situations a rule may uniquely determine the next state without requiring probabilistic candidate selection.

Conceptually:

$$R(C) \rightarrow s_{t+1}$$

where the next state is produced directly from the rule system given the current context $C$.

This capability allows deterministic reasoning processes to coexist naturally with neural inference mechanisms.

For some domains deterministic knowledge may represent the most reliable source of control. Rather than forcing such knowledge to be approximated through learning, the architecture permits it to remain explicitly represented and directly executable.

## 11.3 Hybrid Neuro-Symbolic Control

The most general configuration combines neural prediction, graph traversal, and symbolic control.

In such systems:

- neural inference proposes desirable directions,
- graph structures define admissible transitions,
- rule systems impose symbolic constraints,
- decoding mechanisms perform final candidate selection.

This creates a hybrid neuro-symbolic decision process in which learned and deterministic mechanisms cooperate rather than compete.

The interaction can be summarized as:

$$\text{Context} \rightarrow \text{Vector Inference} \rightarrow \text{Graph Candidates} \rightarrow \text{Rule Filtering} \rightarrow \text{Decoding} \rightarrow \text{Selected State}$$

or, in cases where deterministic knowledge is sufficiently strong,

$$\text{Context} \rightarrow \text{Rule System} \rightarrow \text{Selected State}.$$

A practical example can be found in language-oriented VGPC systems such as GLP–Graph, where states may contain both lexical and grammatical information. Rules can therefore operate on grammatical categories, lexical categories, or combinations of both.

Such rules may enforce additional structural constraints beyond those already present in the graph itself.

In many cases the graph alone may already provide substantial protection against invalid transitions because it is derived from observed training sequences. However, datasets may contain exceptional cases, rare patterns, or undesirable transitions that remain technically reachable. Rule systems provide an additional layer of deterministic control that can prevent such transitions from being selected.

More generally, rule-based control enables explicit human knowledge, symbolic reasoning, and domain expertise to participate directly in the cognitive process. The resulting architecture therefore combines three complementary sources of guidance:

- learned knowledge through neural inference,
- structural knowledge through graph topology,
- explicit knowledge through symbolic rules.

The VGPC framework treats these mechanisms as cooperative components within a single cognitive architecture, allowing systems to balance flexibility, traceability, and deterministic control according to the requirements of the application domain.

# 12 PRE Reasoning Loops

The Propose–Retry–Evaluate (PRE) framework provides the primary mechanism for recursive evaluation, quality control, and self-correction within the VGPC architecture family.

While vector inference, graph traversal, and decoding determine what candidate states or sequences are generated, PRE determines whether the generated result is sufficiently desirable to be accepted.

The framework is intentionally general and can be applied to any cognitive artifact produced by the system, including individual states, state sequences, plans, documents, images, audio structures, actions, memories, or higher-level cognitive constructs.

Unlike architectures that rely solely on prediction, PRE introduces an additional evaluation stage that enables generation to be guided by explicit quality criteria.

$$\text{Generate} \rightarrow \text{Evaluate} \rightarrow \text{Accept or Retry}$$

This simple pattern can be applied recursively at multiple levels of abstraction and with a wide variety of evaluation functions.

## 12.1 Propose–Retry–Evaluate

At its most basic level, PRE consists of three stages.

First, the system proposes a candidate output.

Second, the output is evaluated according to one or more quality measures.

Third, the candidate is either accepted or rejected.

Conceptually:

$$Candidate \rightarrow Evaluator \rightarrow Decision$$

If the candidate satisfies the required acceptance criteria, generation continues.

Otherwise, a new candidate may be generated and evaluated again.

$$Candidate_1 \rightarrow Reject$$

$$Candidate_2 \rightarrow Reject$$

$$Candidate_3 \rightarrow Accept$$

The architecture places no restrictions on how new candidates are obtained.

Alternative candidates may emerge through:

- probabilistic decoding,
- Top-K sampling,
- modified modulation vectors,
- alternative graph traversals,
- stochastic generation mechanisms,
- other domain-specific processes.

An important implication follows immediately.

PRE requires diversity within the generation process.

If inference is completely deterministic and always produces the same output, retry loops cannot improve results because every retry yields an identical candidate.

Consequently, PRE is most effective when multiple plausible candidate trajectories are available.

## 12.2 Recursive Evaluation

The evaluation stage is the central component of PRE.

The architecture intentionally avoids prescribing a particular evaluation mechanism.

Evaluators may range from simple heuristics to highly sophisticated learned systems.

Possible evaluation functions include:

- length constraints,
- structural heuristics,
- grammar validation,
- consistency checks,
- rule compliance,
- classifier-based scoring,
- retrieval-based verification,
- simulation-based validation,
- specialist neural evaluators,
- large language model critics.

The evaluator therefore becomes a highly flexible source of feedback.

In many domains the evaluator may contain knowledge that is difficult to encode directly within the generation process itself.

For example, a grammatical evaluator may be significantly simpler than forcing the generation mechanism to guarantee grammatical correctness in every situation.

Similarly, an LLM-based critic may be capable of assessing coherence, quality, style, or consistency using capabilities that differ substantially from those of the generator.

The VGPC framework therefore treats evaluation as an independent cognitive function rather than an intrinsic property of generation.

## 12.3 Multi-Level Reasoning

One of the most powerful aspects of PRE is its ability to operate across multiple levels of abstraction simultaneously.

Consider a hierarchical generation process:

$$Token \rightarrow Sentence \rightarrow Paragraph \rightarrow Chapter \rightarrow Document$$

A PRE loop may exist at every level.

For example:

- token-level validation,
- sentence-level evaluation,
- paragraph-level coherence checks,
- chapter-level quality assessment,
- document-level consistency evaluation.

This creates a hierarchy of feedback mechanisms that monitor different forms of quality.

Lower levels may focus on local correctness, while higher levels evaluate global consistency and long-range structure.

More generally, PRE levels may evaluate different properties of the same artifact.

For example:

- correctness,
- coherence,
- safety,
- stylistic consistency,
- task completion,
- transition quality,
- behavioral alignment.

The resulting architecture supports a form of recursive reasoning in which generated artifacts are repeatedly inspected and improved before becoming part of the active context.

Although the current formulation emphasizes retry-based correction, the framework also naturally supports refinement-based approaches.

Instead of generating entirely new candidates, evaluator feedback may be used to modify future generation behavior, influence candidate scoring, or steer the system toward improved trajectories.

Consequently, PRE encompasses both iterative retry and iterative refinement strategies.

## 12.4 Quality Control and Self-Correction

PRE transforms evaluation from a passive measurement process into an active control mechanism.

Rather than merely reporting quality, evaluators directly influence which artifacts become part of the evolving state sequence.

Acceptance can be based on:

- fixed thresholds,
- adaptive thresholds,
- ranking criteria,
- comparative evaluation,
- multi-objective scoring functions.

In practice, retry loops are typically bounded.

A system may therefore operate according to a maximum retry budget:

$$R_{max}$$

that limits computational cost.

When all retries are exhausted, several fallback strategies remain possible.

Examples include:

- accepting the highest-scoring candidate,
- accepting the most recent candidate,

- relaxing evaluation thresholds,
- invoking alternative generation mechanisms,
- escalating evaluation to a higher-level subsystem.

These mechanisms ensure that generation remains practical even when perfect solutions cannot be found.

From a cognitive perspective, PRE can be interpreted as a general self-correction loop.

Generation produces hypotheses.

Evaluation assesses their quality.

Retry or refinement mechanisms revise unsuccessful hypotheses.

The process continues until an acceptable result is obtained or resource constraints are reached.

The PRE framework therefore separates generation from judgment in much the same way that graph traversal separates possibility from preference and CAS separates preference from constraint.

Within the VGPC architecture family, PRE serves as a general-purpose reasoning, quality-control, and self-correction mechanism capable of operating across multiple domains and levels of abstraction. It provides an explicit framework for incorporating reflective behavior into intelligent systems while remaining compatible with deterministic, probabilistic, neural, symbolic, and hybrid generation processes.

# 13 Hybrid Intelligence Integration

The Vector–Graph–PRE–CAS (VGPC) architecture family is intentionally designed as a modular cognitive framework rather than a monolithic intelligent system.

The architecture does not assume that a single computational mechanism should be responsible for every aspect of cognition. Instead, it follows a Hybrid Intelligence philosophy in which specialized components cooperate to perform different cognitive functions.

Within this perspective, VGPC systems are viewed as cognitive building blocks that can operate independently or participate as subsystems within larger intelligent ecosystems.

This design reflects a central assumption of the architecture:

*Complex intelligence may emerge from the interaction of complementary cognitive systems rather than from a single homogeneous model.*

The architecture therefore emphasizes modularity, interoperability, inspectability, and compositionality.

## 13.1 VGPC as a Cognitive Subsystem

The VGPC framework is designed to function as a reusable cognitive component.

Rather than defining a complete end-to-end intelligence system, a VGPC instance may specialize in a particular cognitive role.

Possible examples include:

- world modeling,
- memory management,
- planning,
- narrative generation,
- simulation,
- domain-specific reasoning,

- behavior control.

Multiple VGPC systems may coexist within the same architecture and can operate at different abstraction levels.

For example, a high-level planning system may interact with one or more specialized VGPC world models, each responsible for a different domain.

This compositional approach allows complex intelligent behavior to emerge from the interaction of smaller and more tractable subsystems.

An important consequence is that VGPC systems can remain useful even when trained on relatively small datasets and modest computational resources.

Their value derives not solely from scale but also from the explicit structure provided by state spaces, graphs, behavioral control mechanisms, and evaluation loops.

## 13.2 Integration with Large Language Models

The VGPC architecture is not intended as a replacement for large language models.

Instead, both approaches provide complementary capabilities.

Large language models excel at:

- broad generalization,
- language understanding,
- semantic knowledge,
- flexible reasoning,
- natural language generation.

VGPC systems emphasize different properties:

- explicit state management,
- structured world modeling,
- traceability,
- controllability,
- behavioral steering,
- persistent state evolution.

These capabilities naturally complement one another.

A VGPC system may provide structured domain knowledge, world dynamics, behavioral constraints, or memory representations, while a large language model provides linguistic competence and broad semantic reasoning.

Conceptually such a system may be represented as:

$$VGPC + LLM = HybridCognitiveSystem$$

The resulting architecture combines explicit state-space navigation with the generalization capabilities of modern neural language models.

## 13.3 Agentic Architectures

Modern intelligent systems increasingly rely on agentic pipelines composed of multiple interacting components.

The VGPC framework fits naturally within such architectures.

Because state spaces, graphs, CAS vectors, modulation mechanisms, and PRE evaluators are independent components, they can be configured dynamically at runtime according to the requirements of a particular task.

Examples include:

- selecting different CAS profiles,
- activating specialized evaluators,
- switching modulation strategies,
- changing graph traversal policies,
- combining outputs from multiple VGPC systems.

This flexibility allows agentic systems to adapt their behavior without requiring retraining of the entire architecture.

Furthermore, because the neural inference mechanism is only one component of the overall system, its complexity can be scaled independently.

Implementations may employ compact neural models when computational efficiency is important or larger models when additional representational capacity is required.

This scalability enables VGPC systems to function across a wide range of resource budgets while preserving their overall cognitive structure.

## 13.4 World Engines and Memory Systems

One particularly important application of the architecture is the construction of world engines and memory systems.

Because state transitions are represented explicitly, VGPC systems naturally model evolving environments whose internal state changes over time.

The architecture supports both static and dynamic state spaces.

While some domains employ fixed collections of states, others may introduce new states during operation.

For example, a vocabulary may grow over time, new entities may appear within a world model, or memory systems may continuously acquire new information.

This flexibility is possible because neural inference predicts target vectors rather than identifiers tied to a fixed output space.

As long as newly introduced states can be assigned valid vector representations and integrated into the transition graph, they immediately become available to the system.

Consequently, VGPC architectures are naturally compatible with continuous learning scenarios in which knowledge evolves rather than remaining frozen in a single training snapshot.

A domain-specific intelligent system may therefore consist of multiple specialized VGPC instances, one or more large language models, external vector databases, retrieval systems, and agentic orchestration software.

In such environments VGPC components act as structured cognitive modules that maintain explicit state evolution while cooperating with external systems that provide complementary forms of intelligence.

The architecture is therefore best understood not as an isolated model but as a family of interoperable cognitive components designed to participate in larger hybrid intelligent ecosystems.

## 13.5 Example: Autonomous Narration Through Hybrid World Modeling

A concrete example of the Hybrid Intelligence philosophy is provided by the narrator pipeline developed within the Triadic Cosmos ecosystem.

The narrator pipeline combines a VGPC-based world engine with a large language model to produce long-form narrative content with minimal creative input.

Within this architecture, a GLP–Graph system serves as the primary world model. Rather than generating finished prose directly, the world engine maintains an evolving narrative state and produces structured representations of future story developments based on the learned dynamics of its training corpus.

The generated state sequences capture narrative progression, thematic development, character interactions, and world evolution. These state transitions emerge from traversal of the graph-constrained state space and are further influenced by vector inference, CAS steering, and PRE evaluation mechanisms.

A large language model subsequently transforms these structured world states into natural language narratives.

Conceptually the system can be represented as:

$$World\ Engine \rightarrow Story\ State \rightarrow LLM \rightarrow Narrative\ Text$$

or more explicitly:

$$GLP\text{–}Graph \rightarrow Chapter\ Draft \rightarrow LLM \rightarrow Final\ Chapter$$

The resulting architecture illustrates a clear separation of responsibilities.
The VGPC subsystem is responsible for:

- world evolution,
- narrative continuity,
- character consistency,
- long-term state management,
- behavioral steering,
- structural control.

The large language model is responsible for:

- natural language realization,
- semantic expansion,
- stylistic variation,
- linguistic fluency,
- contextual extrapolation.

This division allows each component to operate according to its strengths.

The GLP–Graph system contributes explicit state tracking, controllable narrative progression, and domain-specific structure, while the large language model contributes broad language competence and expressive flexibility.

An important consequence is that substantial narrative emergence can arise from the interaction of both systems.

The world engine provides a constrained narrative trajectory derived from its training corpus, while the large language model extrapolates beyond the explicit training examples during language realization. The resulting stories therefore exhibit characteristics inherited from both the learned narrative canon and the broader semantic knowledge encoded within the language model.

The narrator pipeline demonstrates how VGPC architectures can function as specialized cognitive subsystems within larger hybrid intelligent systems. Rather than replacing large language models, the architecture enriches them with explicit world dynamics, structured state evolution, and controllable generative behavior.

More generally, the example illustrates the central thesis of Hybrid Intelligence: complex behavior can emerge from the cooperation of multiple specialized cognitive components, each contributing capabilities that are not easily obtained from a single monolithic model.

# 14 Advantages and Limitations

As a cognitive architecture, VGPC occupies a different position within the design space of intelligent systems than contemporary large-scale transformer architectures. The framework therefore exhibits a distinct set of strengths, limitations, and trade-offs.

Importantly, the architecture is not proposed as a universal replacement for existing approaches. Rather, it represents an alternative set of design priorities centered around structured state spaces, explicit control mechanisms, modular composition, and hybrid intelligent systems.

The value of the architecture should therefore be evaluated in terms of the problems it attempts to solve rather than by direct comparison with systems optimized for unrestricted large-scale statistical learning.

## 14.1 Strengths

The VGPC architecture provides several advantages that arise from its explicit state-space representation and modular design.

A first advantage is traceability.

Because generation occurs through explicit state transitions, graph traversal, and observable decoding mechanisms, the reasoning process remains largely inspectable. Generated outputs can often be traced back to specific states, transitions, constraints, and evaluative decisions.

A second advantage is controllability.

Behavior can be influenced through multiple independent mechanisms including graph topology, rule systems, vector modulation, CAS steering, and PRE evaluation loops. This layered approach provides substantially more explicit control than architectures that rely exclusively on learned parameter spaces.

A third advantage is modularity.

The architecture naturally decomposes into independent subsystems that may be modified, replaced, or extended without requiring fundamental redesign of the overall system. This property supports experimentation and facilitates integration into larger hybrid intelligent environments.

A fourth advantage is computational accessibility.

VGPC systems can be trained and deployed on comparatively small datasets and modest computational resources. Since the architecture incorporates explicit structure through graph representations and deterministic components, useful behavior can emerge without requiring extremely large parameter counts or massive training corpora.

A fifth advantage is adaptability through continuous evolution.

New states can be introduced without fundamentally changing the output dimensionality of the neural inference mechanism. Because inference predicts vectors rather than identifiers tied to a fixed vocabulary, state spaces may grow over time as new concepts, memories, entities, or knowledge are acquired.

This property makes the architecture naturally compatible with continuous learning systems.

A sixth advantage is what may be described as *canon compression*.

Rather than attempting to store broad statistical representations of the entire training distribution, VGPC systems can capture characteristic transition structures and attractor dynamics within a comparatively compact state-space representation. This can be particularly useful for domain-specific cognitive systems that must preserve a consistent internal canon or behavioral identity.

Finally, the architecture supports horizontal scaling.

Instead of constructing ever larger monolithic models, complex capabilities may emerge through the cooperation of multiple specialized VGPC systems, each focused on a distinct domain or cognitive function.

## 14.2 Limitations

Despite these strengths, the architecture also introduces several limitations.

The most obvious limitation arises from its dependence on structured state spaces.

Graphs, state representations, rule systems, and evaluation mechanisms must either be constructed explicitly or learned indirectly. For some domains this may require significant engineering effort.

A second limitation concerns scalability.

Transformer architectures have demonstrated remarkable performance when trained at extreme scales with vast quantities of data and computation. At present, there is no evidence that VGPC architectures exhibit scaling properties that allow them to compete directly with frontier foundation models along the same trajectory.

The architecture therefore should not be viewed as a drop-in replacement for large-scale transformer systems.

A third limitation concerns state representation.

The usefulness of vector inference and graph traversal depends heavily on the quality of the underlying state definitions. Poorly chosen state spaces may restrict expressiveness or introduce suboptimal transition structures.

A fourth limitation concerns graph construction and maintenance.

As domains become larger and more complex, the structure of the transition space may itself become increasingly difficult to manage.

Although procedural and dynamically generated graphs mitigate this issue, they do not eliminate it completely.

A fifth limitation arises from evaluation costs.

PRE loops can significantly improve quality but may increase computational requirements, particularly when sophisticated evaluators or multiple recursive levels are employed.

Finally, the architecture benefits most when meaningful state transitions exist.

Domains that are naturally represented as state-space trajectories align well with the framework, whereas domains lacking useful state abstractions may prove less suitable.

## 14.3 Trade-Offs

The VGPC architecture intentionally makes trade-offs that differ from those found in contemporary transformer systems.

The architecture generally favors:

- traceability over opacity,
- explicit structure over implicit representation,
- controllability over unrestricted emergence,
- modularity over monolithic design,
- continuous evolution over fixed training snapshots,

- specialized subsystems over universal models.

These trade-offs should not be interpreted as universally superior choices.

Rather, they reflect different architectural priorities.

For example, modern large language models excel at large-scale statistical generalization and broad knowledge acquisition. VGPC systems instead emphasize structured cognitive processes, explicit state management, and deterministic control mechanisms.

Similarly, many neural architectures rely on statistical probability distributions over output spaces. VGPC systems can instead be interpreted as operating through attractor dynamics within graph-constrained state spaces, where candidate states compete based on vector similarity, behavioral steering, and evaluative feedback.

Neither paradigm is inherently superior.

They address different aspects of intelligent behavior and can often be viewed as complementary rather than competing approaches.

Indeed, one of the central hypotheses underlying the Hybrid Intelligence methodology is that future intelligent systems may benefit from combining both paradigms. Large-scale neural models can provide broad generalization and semantic flexibility, while VGPC systems contribute explicit structure, persistent state evolution, traceability, and domain-specific control.

From this perspective, the most promising application of the architecture may not be as an isolated intelligent system, but as a specialized cognitive component within larger hybrid ecosystems composed of complementary architectural paradigms.

# 15 Conclusion

This paper introduced the *Vector–Graph–PRE–CAS (VGPC) Cognitive Architecture Family*, a framework for constructing modular cognitive systems that combine vector-based representations, graph-constrained state spaces, neural inference, recursive evaluation mechanisms, behavioral steering vectors, and optional rule-based reasoning.

The central objective of the architecture is not to maximize unrestricted emergence, but to balance adaptability with controllability. By separating state representation, state-space structure, inference, evaluation, and behavioral control into distinct architectural components, VGPC enables the construction of intelligent systems that remain inspectable, traceable, and modular while retaining the benefits of learned representations.

The architecture occupies an intermediate position between classical symbolic systems and modern neural approaches. It incorporates neural learning where appropriate while preserving explicit state spaces, deterministic transition mechanisms, and evaluative control processes. As a result, the framework can be viewed as a neuro-symbolic cognitive architecture designed around controlled state-space navigation rather than unrestricted predictive generation.

Rather than introducing an isolated computational model, the VGPC framework generalizes principles that emerged during the development of concrete systems such as GLP–Graph. The resulting architectural family provides a common foundation upon which multiple implementations, world engines, reasoning engines, memory systems, and domain-specific cognitive subsystems can be built.

The framework is equally compatible with standalone applications and hybrid intelligent systems. In particular, VGPC architectures may serve as structured cognitive components within larger agentic environments that incorporate large language models, external memory systems, planning systems, and specialized reasoning modules. Such integrations align with the broader Hybrid Intelligence methodology, where intelligence emerges from the interaction of multiple complementary subsystems rather than from a single homogeneous mechanism.

Future work will explore specific instances of the architecture family, including extensions of the GLP–Graph architecture, multi-granularity state representation mechanisms, advanced modulation techniques, recursive PRE-based reasoning systems, and integrations with autonomous hybrid narrator and

agent pipelines. These implementations will provide concrete demonstrations of how the general principles described in this paper can be applied within practical cognitive systems.

The primary contribution of this work is therefore not a particular model, algorithm, or application, but the formalization of a broader architectural family that offers a structured design space for future research into modular, inspectable, and hybrid intelligent systems.

# A Triadic Cosmos Ecosystem Context

The present paper is part of the broader *Triadic Cosmos* ecosystem: a unified research program that develops a compositional ontology of intelligence together with modular, inspectable computational architectures. The ecosystem spans conceptual theory, executable reference implementations, and fully documented empirical demonstrations, forming a coherent framework for studying hybrid intelligent systems.

All materials, extended notes, and evolving documentation are publicly available in the open GitHub project:

`https://github.com/triadic-cosmos`

# References


[1] J. De Witte, "A Safe and Modular AGI Architecture," Triadic Cosmos Ecosystem, 2026.

[2] J. De Witte, "Architectural Intelligence Theory: A Unification of Cognition," Triadic Cosmos Ecosystem, 2026.

[3] J. De Witte, "Language as Graph Traversal: The DMLG Architecture," Triadic Cosmos Ecosystem, 2026.

[4] J. De Witte, "Learnable Lemma Embeddings for GLP Models," Triadic Cosmos Ecosystem, 2026.

[5] J. De Witte, "The GLP–Graph Architecture: Combining GLP–Mono with DMLG," Triadic Cosmos Ecosystem, 2026.

[6] J. De Witte, "Extending GLP–Graph with CAS: Architectural Personality Modulation," Triadic Cosmos Ecosystem, 2026.

[7] J. De Witte, "A Lightweight Autonomous Narrator Pipeline," Triadic Cosmos Ecosystem, 2026.